

1 Изучение предметной области и анализ требований к LMS-платформам

- **1.1 Анализ предметной области: структура и участники учебного процесса в высшем учебном заведении**
- 1.2 Обзор и сравнительный анализ существующих систем управления обучением (LMS)
- 1.3 Анализ лучших практик организации интерфейсов и пользовательского опыта в образовательных сервисах
- 1.4 Разработка функциональных требований к клиентской части платформы
- 1.5 Разработка нефункциональных требований (*производительность, масштабируемость, безопасность, интернационализация, доступность*)
- **1.6 Проектирование и описание пользовательских сценариев**

1.1 Анализ предметной области: структура и участники учебного процесса в высшем учебном заведении

Проектирование и разработка клиентской части платформы для обучения студентов требуют глубокого анализа структуры предметной области и формализации протекающих в ней процессов. Для построения масштабируемой программной системы на начальном этапе необходимо решить задачу трансляции реальных объектов и связей академической среды вуза в строгие термины программной инженерии. Это позволяет сформировать единый понятийный аппарат (Ubiquitous Language) для исключения семантического разрыва между реальными бизнес-процессами и их программной абстракцией в коде клиентского приложения.

1.1.1 Терминологический глоссарий предметной области

В рамках автоматизируемой образовательной среды ключевые сущности учебного процесса формализуются следующим образом:

- **Электронный учебный курс (Course):** структурированная единица образовательного контента, выступающая в качестве контейнера для учебных материалов, метаданных и связей с пользователями. На уровне клиентского приложения курс представляется в виде иерархического дерева разделов.
- **Элемент курса (Course Item):** минимальная неделимая единица учебного контента. Сущность является полиморфной и на клиенте разделяется на типы: текстовая лекция, видеоматериал, PDF-документ, презентация, интерактивный тест или практическое задание.
- **Банк вопросов (Question Bank):** изолированное логическое хранилище тестовых и контрольных заданий, категоризованных по темам, уровням сложности и типам ответов. Используется преподавателем для динамической или статической сборки тестов.
- **Балльно-рейтинговая система (БРС):** накопительная модель оценки успеваемости студентов, требующая от клиентского интерфейса вывода динамических аналитических дашбордов успеваемости и интерактивных ведомостей для преподавателей.
- **Академическая группа (Academic Group):** сплоченный коллектив пользователей с ролью «Студент», объединенных единым учебным годом поступления и направлением подготовки, проходящих обучение по единой сетке учебного плана.

1.1.2 Организационно-структурная модель и ее отображение в SPA-архитектуре

Организационная структура университета имеет строго иерархический характер. С точки зрения информационного моделирования, вложенность сущностей вуза имеет следующий вид:

Университет→Институт / Факультет→Кафедра→Направление
подготовки→Академическая группа→Студент

Университет→Институт / Факультет→Кафедра→Направление
подготовки→Академическая группа→Студент

Данная иерархия напрямую проецируется на архитектуру Single Page Application (SPA), определяя структуру клиентского роутинга и интерфейсные паттерны:

- **Динамический клиентский роутинг (Routing):** иерархия вуза задает вложенную структуру путей в менеджере маршрутов `vue-router`. Например, страница администрирования конкретной группы описывается динамическим маршрутом вида `/admin/institutes/:instituteId/departments/:departmentId/groups/:groupId`. Такая структура гарантирует логичность переходов и позволяет легко извлекать идентификаторы сущностей из параметров пути для выполнения соответствующих API-запросов.
- **Древовидные компоненты интерфейса (Tree Views):** для визуализации иерархии в панели Администратора используются интерактивные древовидные компоненты (например, `<Tree />` из библиотеки PrimeVue). Это позволяет пользователю наглядно воспринимать связи между кафедрами, направлениями и группами, а также выполнять операции добавления, перемещения или удаления узлов дерева в рамках единого экрана.
- **Каскадный выбор (Cascade Selects):** при создании учетных записей или закреплении преподавателей за группами используются каскадные выпадающие списки (например, `<CascadeSelect />`). Клиентское приложение динамически фильтрует списки дочерних элементов при выборе родительских (выбор института ограничивает список доступных кафедр, а выбор кафедры — список направлений), снижая вероятность ошибок ввода.

1.1.3 Жизненный цикл и состояния ключевых процессов

Учебный процесс на платформе не является статичным и представляет собой совокупность динамических процессов, которые клиентское приложение должно визуализировать в зависимости от их текущего состояния. Для этого поведение ключевых

сущностей описывается в виде конечных автоматов (Finite State Machines), состояния которых управляют доступностью элементов интерфейса.

- **Черновик (Draft):** начальное состояние курса при создании. Курс доступен для редактирования и наполнения Преподавателю, но полностью скрыт от отображения в интерфейсах Студентов.
- **Активный (Active):** курс опубликован. Клиентское приложение делает его видимым на дашборде студентов прикрепленных групп, открывая доступ к последовательному изучению тем и прохождению тестов.
- **Архивный (Archived):** курс переведен в архив. Для Студентов курс переходит в режим «только чтение», убирается с главного дашборда активных дисциплин и перемещается в архивный раздел.
- **Не начато (Not Started):** задание доступно для ознакомления, кнопка отправки решения активна.
- **В процессе выполнения (In Progress):** Студент начал работу над заданием (например, сохранил черновик ответа).
- **Отправлено на проверку (Pending Review):** решение студента отправлено на сервер. На клиенте у Студента блокируется возможность повторной отправки или редактирования файлов (интерфейс переходит в режим ожидания), а у Преподавателя в ведомости данная запись подсвечивается как требующая ручной проверки.
- **Проверено (Graded):** Преподаватель выставил оценку. Студент видит в своем интерфейсе баллы и комментарий, редактирование задания полностью блокируется для обеих сторон.
- **Доступен (Available):** тест опубликован в рамках активного курса, кнопка запуска тестирования активна для Студента.
- **Выполняется (Active Test Session):** Студент запустил тест. Клиентское приложение переходит в полноэкранный режим фокусировки внимания, запускается таймер обратного отсчета. Текущее состояние ответов периодически кэшируется в локальном хранилище (`LocalStorage`) для предотвращения потери прогресса при случайной перезагрузке страницы или сбое сети.
- **Завершен (Submitted):** время вышло или студент нажал кнопку завершения. Данные отправляются на сервер, запускается процесс автоматического оценивания, интерфейс Студента переключается на экран отображения результатов.

Таким образом, формализация предметной области в рамках подраздела 1.1 позволяет четко определить структуру будущих моделей данных на клиенте, спроектировать логику

динамического роутинга и описать реактивное поведение интерфейса в зависимости от состояний ключевых бизнес-процессов платформы.

1.2 Обзор и сравнительный анализ существующих систем управления обучением (LMS)

Для определения вектора разработки и выявления наиболее эффективных технических и интерфейсных решений необходимо провести критический обзор существующих систем управления обучением (LMS). На сегодняшний день в академической среде России и за рубежом доминируют как классические монолитные платформы с открытым исходным кодом, так и современные облачные SaaS-решения.

Наиболее распространенной платформой в отечественных вузах является Moodle (Modular Object-Oriented Dynamic Learning Environment). Это мощная монолитная система с открытым исходным кодом, написанная на языке PHP. Главным достоинством Moodle выступает развитая экосистема плагинов и полная поддержка сложных академических сценариев (учет групп, ведомости, интеграция с деканатом). Однако с точки зрения фронтенд-архитектуры Moodle имеет серьезные недостатки. В системе используется классический серверный рендеринг (SSR) на основе шаблонизатора Mustache и устаревших JS-библиотек (jQuery, RequireJS, YUI). Такой подход приводит к избыточной нагрузке на сеть при каждом клике пользователя, медленному переключению между страницами и перегруженному DOM-дереву. Реализация плавных анимаций, интерактивного тестирования в реальном времени и удобной мобильной версии в рамках такой архитектуры сопряжена с колоссальными трудозатратами, что негативно сказывается на пользовательском опыте студентов.

Зарубежным аналогом Moodle с открытым кодом является Canvas LMS от компании Instructure. Платформа построена на базе веб-фреймворка Ruby on Rails, а в качестве фронтенд-стека использует библиотеку React. Canvas предлагает более отзывчивый и визуально чистый интерфейс по сравнению с Moodle, однако его клиентская часть глубоко интегрирована в монолит бэкенда, что усложняет кастомизацию. Самостоятельное развертывание и локальная кастомизация Canvas LMS представляют собой сложную инженерную задачу из-за жесткой привязки к облачной инфраструктуре AWS и ориентации на SaaS-модель для западного рынка.

Среди отечественных разработок выделяется платформа Stepik — современное облачное решение, ориентированное на интерактивные курсы и автоматическую проверку задач (в том числе по программированию). Клиентская часть Stepik реализована в виде высокопроизводительного Single Page Application (SPA) на базе React. Платформа предлагает эталонный пользовательский опыт: мгновенный переход по страницам, плавно работающие таймеры тестов и удобный конструктор шагов курса.

Однако Stepik поставляется исключительно по модели SaaS в закрытом облаке. Это делает невозможным его развертывание на локальных серверах университета (on-premise) для обеспечения безопасности персональных данных и интеграции с внутренними базами данных вуза. Кроме того, в Stepik отсутствуют гибкие инструменты администрирования сложной вузовской структуры (учет кафедр, учебных направлений и закрепленных за ними академических групп).

Для систематизации результатов обзора и наглядного сопоставления архитектурных особенностей клиентских частей рассмотренных платформ составлена сравнительная таблица.

Таблица 2.

Сравнительный анализ архитектуры и UX существующих LMS-платформ

Критерий сравнения Moodle Canvas LMS Stepik

Архитектура клиента	SSR (PHP)	Гибридная (Rails + React)	SPA (React)
Время отклика (UX)	Низкое (полный перезапуск глав)	Среднее	Высокое (без перезапуска)
Адаптивность	Удовлетворительная	Хорошая	Отличная
Возможность локального развертывания	Полная	Затрудненная	
Отсутствует			
Поддерживаемая вузовская структура	Полная	Полная	Ограниченная

Проведенный сравнительный анализ существующих LMS-систем позволяет выделить наиболее удачные инженерные и интерфейсные практики. В частности, высокая интерактивность и отзывчивость SPA-архитектуры Stepik, а также глубина проработки ролевых моделей и иерархических структур Moodle служат ключевыми ориентирами для проектирования образовательных систем.

При этом целью данной работы и совместной деятельности команды разработчиков не является создание альтернативы или прямая замена зрелых коммерческих и академических решений. Основная научно-практическая задача заключается в систематическом изучении и прохождении полного жизненного цикла разработки сложной, высоконагруженной образовательной веб-платформы с нуля.

В рамках данного исследования критически важным этапом выступает проектирование такой архитектуры клиентского приложения, которая не только обеспечит базовую

функциональность (MVP), но и заложит надежный фундамент для легкого, безопасного и структурированного расширения возможностей платформы в будущем.

Использование базового фреймворка Vue 3 в сочетании с методологией Feature-Sliced Design (FSD) призвано решить именно эту задачу. Строгая декомпозиция кодовой базы по слоям и сегментам позволит сторонним разработчикам и будущим командам без труда внедрять новые функциональные модули в платформу, не нарушая целостности ее существующей архитектурной структуры.

1.4 Разработка функциональных требований к клиентской части платформы

Функциональные требования определяют то, какие возможности должна предоставлять система пользователям. В ходе анализа были формализованы и систематизированы функциональные требования для трех ключевых ролей пользователей: Администратора, Преподавателя и Студента.

Ниже приведено описание функциональных требований к платформе к интерфейсу каждого участника образовательного процесса.

Общие функциональные требования

Перед началом работы с платформой клиентская часть должна обеспечивать прохождение сквозных процедур, обязательных для всех категорий пользователей:

- **Идентификация и аутентификация:** каждый пользователь системы должен проходить процедуру авторизации с помощью пары уникального логина и пароля.
- **Разграничение прав доступа:** после успешного входа система определяет назначенную роль пользователя и на клиентском уровне динамически формирует доступный набор интерфейсов, меню навигации и защищенных маршрутов.
- **Сброс и восстановление пароля:** клиентская часть должна содержать сценарий самостоятельного восстановления пароля пользователем через указание электронной почты, связанной с учетной записью, и последующее подтверждение изменений.

Функциональные требования к интерфейсу Администратора

Интерфейс Администратора ориентирован на техническое обеспечение работоспособности системы, управление иерархической структурой вуза и ведение базы данных пользователей:

- **Управление учетными записями пользователей:** Администратор должен иметь возможность создавать новые учетные записи (как вручную для единичных пользователей, так и пакетно посредством импорта файлов в формате Excel), редактировать существующие профили (изменять электронную почту, переназначать группы) и удалять неактуальные аккаунты.
- **Управление иерархической структурой университета:** интерфейс должен предоставлять инструменты для визуального редактирования древовидной структуры направлений подготовки, институтов/факультетов, кафедр и академических групп.
- **Управление правами доступа преподавателей:** Администратор закрепляет за конкретным преподавателем список доступных ему учебных направлений, институтов или групп, определяя его зону ответственности в системе.

Функциональные требования к интерфейсу Преподавателя

Интерфейс Преподавателя предназначен для создания учебного контента, контроля успеваемости студентов и управления учебными курсами:

- **Управление курсами:** возможность создания новых курсов с указанием названия, описания и закрепленных академических групп студентов. Преподаватель может переводить курсы в архивный статус и добавлять других преподавателей в качестве соавторов.
- **Конструирование содержания:** интерфейс позволяет выстраивать структуру курса и наполнять ее учебными материалами разных форматов (текст, презентации, PDF-документы, видеофайлы).
- **Управление банком вопросов:** создание и ведение централизованной базы вопросов различных типов (тестовые с одиночным или множественным выбором, контрольные со свободным кратким или развернутым ответом). Вопросам могут назначаться параметры сложности.
- **Создание тестов и контрольных заданий:** сборка тестов из готовых банков вопросов, установка проходного балла, ограничения по времени на прохождение и жестких сроков сдачи.
- **Мониторинг успеваемости студентов:** Преподавателю предоставляется детальная статистика по курсу в виде интерактивных графиков и таблиц, отражающих прогресс прохождения тем, результаты тестирования и общую

успеваемость групп. Должен быть реализован экспорт этих отчетов в файлы формата Excel.

- **Проверка работ:** интерфейс должен содержать удобный инструмент для ручной проверки развернутых ответов студентов на контрольные задания с возможностью выставления оценки и отправки текстового комментария.

Функциональные требования к интерфейсу Студента

Интерфейс Студента представляет собой адаптивное рабочее пространство, спроектированное для максимально комфортного и сфокусированного прохождения обучения:

- **Дашборд доступных курсов:** Студент должен видеть наглядный перечень всех учебных курсов, на которые он записан.
- **Взаимодействие с учебным материалом:** навигация по структуре курса, последовательное чтение лекций, просмотр прикрепленных видео, а также предпросмотр и скачивание методических материалов (PDF, презентаций).
- **Прохождение тестовых работ:** интерактивное выполнение тестов (с визуальным отображением оставшегося времени) и отправка решений практических или контрольных работ (включая загрузку файлов с выполненными заданиями).
- **Мониторинг успеваемости:** доступ к разделу с собственными оценками по всем элементам курса, обратной связью от преподавателей и общим индикатором прогресса прохождения учебной программы.

1.5 Разработка нефункциональных требований

Нефункциональные требования определяют качественные характеристики клиентской части платформы, накладывая ограничения на архитектурные решения, производительность, безопасность и пользовательский опыт. Специфика разработки клиентского SPA-приложения для академической среды обуславливает следующие нефункциональные требования:

Требования к интерфейсу пользователя

- **Интуитивность и доступность:** интерфейс должен быть интуитивно понятным для пользователей с разным уровнем компьютерной грамотности. Навигация должна строиться по принципу минимального количества кликов для достижения целевого действия.
- **Кроссплатформенная адаптивность:** клиентская часть должна корректно отображаться и сохранять полную функциональность на широком спектре устройств: персональные компьютеры, ноутбуки, планшеты.

- **Доступность:** обязательна поддержка стандартов доступности для людей с ограниченными возможностями здоровья (контрастные цветовые схемы, масштабирование текста без разрушения верстки, поддержка навигации с клавиатуры).

Производительность и оптимизация (Performance)

- **Время первичной загрузки:** размер исходного бандла приложения должен быть минимизирован за счет разделения кода и ленивой загрузки некритичных модулей. Время до первой отрисовки контента (FCP) при стандартном интернет-соединении должно составлять не более 1.5–2 секунд.
- **Отзывчивость интерфейса:** переходы между страницами внутри Single Page Application должны происходить плавно, без визуального мерцания и полной перезагрузки страниц. Для сглаживания времени ожидания ответов от сервера необходимо использовать анимированные скелетоны.
- **Управление состоянием:** хранение глобального состояния приложения должно исключать лишние повторные сетевые запросы при переключении между вкладками или возврате на предыдущие страницы.

Безопасность и разграничение доступа

- **Защита каналов связи:** весь обмен данными между клиентской и серверной частями платформы должен осуществляться по защищенному протоколу HTTPS с использованием современных алгоритмов шифрования.
- **Безопасное хранение сессий:** авторизационные токены (JWT) должны храниться в защищенных клиентских хранилищах, минимизирующих риски XSS-атак (рекомендуется использование HttpOnly Cookie или защищенного стейта приложения).
- **Строгая изоляция данных:** на клиентском уровне должна быть жестко заблокирована возможность просмотра данных, не принадлежащих текущему пользователю. Студент может видеть только свои оценки, а Преподаватель — только данные студентов, привязанных к его курсам. Изменение персональных данных пользователей и иерархии вуза должно быть полностью недоступно в интерфейсе Преподавателя и Студента.

Надежность и отказоустойчивость

- **Обработка сетевых ошибок:** в случае кратковременного обрыва связи или недоступности API бэкенда клиентская часть должна корректно обрабатывать эти ситуации (выводить понятные пользователю предупреждения с предложением

повторить попытку), предотвращая зависание интерфейса или потерю несанкционированно введенных данных (например, ответов на вопросы теста).

- **Автоматическое сохранение прогресса:** при прохождении студентом тестирования его промежуточные ответы должны автоматически кэшироваться или отправляться на сервер, чтобы в случае случайного закрытия вкладки или сбоя компьютера студент мог продолжить выполнение задания с того же места.

1.6 Проектирование и описание пользовательских сценариев

Для детальной спецификации функциональных требований и последующего проектирования интерфейсов необходимо формализовать основные сценарии взаимодействия пользователей с платформой. Каждый сценарий описывает последовательность действий актора и реакцию системы, определяя прецеденты использования клиентской части приложения.

1.6.1 Сценарии взаимодействия для роли «Администратор»

Сценарий 1.6.1.1: Ручное создание учетной записи пользователя

- **Предусловия:** Администратор авторизован в системе и находится в административной панели.
- **Основной сценарий (Шаги):**
 1. Администратор переходит на страницу «Управление пользователями».
 2. Нажимает кнопку «Создать пользователя».
 3. Заполняет интерактивную форму ввода данных (ФИО, уникальный логин, email, выбор роли: Администратор, Преподаватель, Студент).
 4. Нажимает кнопку «Сохранить».
- **Результат:** Данные отправляются на сервер, в базе данных создается запись, а клиентская часть отображает уведомление об успешном создании.

Сценарий 1.6.1.2: Импорт учетных записей из внешнего файла

- **Предусловия:** Администратор подготовил файл со списком пользователей в формате Excel.
- **Основной сценарий (Шаги):**
 1. Администратор переходит в раздел «Управление пользователями».
 2. Нажимает кнопку «Импортировать из файла».
 3. В диалоговом окне выбирает подготовленный файл и нажимает «Загрузить»..
 4. Администратор подтверждает действие нажатием кнопки «Импортировать».
- **Результат:** Пакет данных отправляется на сервер, в системе регистрируются новые пользователи, интерфейс выводит отчет об успешном импорте и количестве созданных записей.

Сценарий 1.6.1.3: Модификация организационной структуры вуза

- **Предусловия:** Администратор находится в разделе «Иерархия структуры».
- **Основной сценарий (Шаги):**
 1. Администратор просматривает древовидную структуру направлений подготовки, институтов и групп.
 2. Выбирает необходимый узел дерева (например, конкретный Институт) и нажимает кнопку «Добавить дочерний элемент» или «Редактировать».
 3. Вводит новое название (например, название создаваемой академической группы) и сохраняет изменения.
- **Результат:** Дерево структуры динамически перерисовывается на клиенте, отправляя обновленную конфигурацию иерархии на сервер.

Сценарий 1.6.1.4: Просмотр и редактирование профиля пользователя

- **Предусловия:** Пользователь зарегистрирован в системе.
- **Шаги:**
 1. Администратор переходит на страницу «Пользователи».
 2. Использует фильтры и строку поиска для нахождения нужного человека.
 3. Кликает на имя пользователя для открытия карточки профиля.
 4. Вносит необходимые изменения (корректирует email, переназначает учебную группу или направление).
 5. Нажимает кнопку «Сохранить изменения».
- **Результат:** Обновленная информация отправляется на сервер, локальный стек данных на клиенте обновляется, выводится уведомление об успешном сохранении.

1.6.2 Сценарии взаимодействия для роли «Преподаватель»

Сценарий 1.6.2.1: Создание учебного курса

- **Предусловия:** Преподаватель авторизован в системе.
- **Шаги:**
 1. На главной странице нажимает кнопку «Добавить курс».
 2. Вводит название и подробное описание курса.
 3. При необходимости указывает соавторов курса из числа других преподавателей.
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** Курс создается и становится доступным для редактирования и наполнения в личном кабинете преподавателя.

Сценарий 1.6.2.2: Структурирование содержания курса

- **Предусловия:** Учебный курс успешно создан.
- **Шаги:**
 1. Преподаватель переходит в раздел «Содержание курса».
 2. Нажимает кнопку добавления новой темы (например, «Раздел 1»).
 3. Внутри созданной темы нажимает «Добавить подтему».
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** На клиенте формируется иерархическая структура курса, готовая к наполнению материалами.

Сценарий 1.6.2.3: Добавление учебных материалов в курс

- **Предусловия:** Структура курса (разделы и подразделы) сформирована.
- **Шаги:**

1. Преподаватель выбирает нужную подтему в структуре курса.
 2. Нажимает кнопку «Добавить материал».
 3. Выбирает тип материала (видео, презентация, PDF-документ, текстовый файл) и загружает его в систему.
 4. Указывает название материала и краткое описание, после чего нажимает «Сохранить».
- **Результат:** Материал прикрепляется к подтеме курса и становится доступен для изучения студентам.

Сценарий 1.6.2.4: Добавление вопросов в банк вопросов

- **Предусловия:** Преподаватель находится в разделе «Банк вопросов».
- **Шаги:**
 1. Выбирает существующий банк из списка или создает новый нажатием на «Добавить банк вопросов».
 2. Внутри выбранного банка нажимает кнопку «Добавить вопрос».
 3. Определяет тип вопроса (тестовый с вариантами, контрольный с текстовым вводом).
 4. Заполняет текст вопроса и указывает сложность.
 5. Для тестового вопроса добавляет варианты ответов и отмечает правильные; для программного — указывает критерии оценки и тесты.
 6. Нажимает «Сохранить вопрос».
- **Результат:** Вопрос заносится в базу данных банка вопросов и готов к использованию в тестах.

Сценарий 1.6.2.5: Создание теста на основе банка вопросов

- **Предусловия:** Создана подтема курса, сформирован и наполнен банк вопросов.
- **Шаги:**
 1. Преподаватель переходит в нужную подтему курса и нажимает «Добавить тест».
 2. Вводит название теста, описание и параметры прохождения (время выполнения, дедлайны).
 3. Переходит во вкладку «Вопросы» и нажимает «Выбрать вопросы из банка».
 4. Фильтрует вопросы по типу и сложности, отмечает галочками нужные и импортирует их.
 5. Настраивает количество начисляемых баллов за каждый верный ответ.
 6. Нажимает «Сохранить тест».
- **Результат:** Тест публикуется в структуре курса и становится доступен для прохождения студентам.

Сценарий 1.6.2.6: Проверка контрольных заданий студентов

- **Предусловия:** Студенты выполнили контрольные работы, требующие ручной оценки.
- **Шаги:**
 1. Преподаватель переходит в раздел статистики и успеваемости курса.
 2. Выбирает необходимое задание из списка требующих проверки.
 3. Просматривает прикрепленные файлы решений студентов.
 4. Выставляет оценку в баллах (вручную) и при необходимости пишет комментарий с разбором ошибок.
 5. Нажимает «Сохранить оценки».
- **Результат:** Оценки фиксируются в системе, студент получает уведомление, а общая ведомость успеваемости автоматически пересчитывается.

Сценарий 1.6.2.7: Отслеживание успеваемости и экспорт отчетов

- **Предусловия:** Студенты начали прохождение курса.
- **Шаги:**
 1. Преподаватель переходит в раздел статистики курса.
 2. Выбирает академическую группу или конкретного студента для анализа.
 3. Изучает сводный прогресс прохождения тем и результаты сданных тестов.
 4. Нажимает кнопку «Экспорт».
 5. Выбирает желаемый формат файла (CSV или Excel) и скачивает его на устройство.
- **Результат:** На устройство скачивается файл с актуальными данными об успеваемости студентов.

Сценарий 1.6.2.8: Назначение учебных групп на курс

- **Предусловия:** Курс создан, Администратор назначил преподавателя на соответствующие группы.
- **Шаги:**
 1. Преподаватель заходит в настройки нужного курса.
 2. Переходит во вкладку «Доступ групп».
 3. Из предложенного перечня доступных ему групп отмечает галочками те, для которых открывается данный курс.
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** Курс появляется в личных кабинетах всех студентов выбранных академических групп.

1.6.3 Сценарии взаимодействия для роли «Студент»

Сценарий 1.6.3.1: Прохождение разделов учебного курса

- **Предусловия:** Студент авторизован на платформе и записан на курс.
- **Шаги:**
 1. Студент открывает список своих курсов и переходит на нужный курс.
 2. Система выводит структуру доступных материалов.
 3. Студент выбирает лекцию или практический материал для изучения.
 4. В зависимости от типа контента: изучает текст на странице, просматривает видеоматериал или скачивает прикрепленный PDF-документ (презентацию).
 5. Использует систему навигации для быстрого перехода к следующей теме курса.
- **Результат:** В системе фиксируется факт изучения темы, прогресс прохождения курса студентом увеличивается.

Сценарий 1.6.3.2: Прохождение тестовой работы

- **Предусловия:** Студенту доступен опубликованный в курсе тест.
- **Шаги:**
 1. Студент открывает список материалов курса и выбирает активный тест.
 2. Ознакомляется с параметрами (количество попыток, время) и нажимает кнопку начала тестирования.
 3. Последовательно отвечает на вопросы, выбирая варианты ответов или вписывая краткие ответы с клавиатуры.
 4. Контролирует оставшееся время по таймеру на экране.
 5. Нажимает кнопку «Завершить тест».
- **Результат:** Система фиксирует ответы, отправляет их на автоматическую проверку и (в зависимости от настроек теста) сразу выводит набранные баллы и детальный отчет по ошибкам.